

A proposed interface for `UnitTestDesign.jl`

This is Astra's best guess at an interface, intended to be compared with other proposals. It describes proposed behavior, not the current implementation. The examples are interface sketches; they are not executable against today's package.

Purpose and scope

The package should help someone answer:

I have several choices to test together. Which cases will exercise their interactions without running every possible configuration?

The strongest application is a test whose configurations are meaningful and whose execution is expensive enough to justify deliberate selection. Exhaustive loops remain a good choice for small, cheap spaces. Randomized and property-based tests remain useful for exploring concrete inputs. This interface should fit inside those approaches rather than require adopting a new testing framework.

The proposal does not assume that a larger feature set will create demand. Its central bet is that named choices, understandable constraints, and inspectable coverage make the existing algorithms useful with little additional work.

The author supplies three things:

1. Finite choices, with names that mean something in the problem domain.
2. Rules describing which choices can occur together, when needed.
3. Ordinary Julia code to construct inputs and check behavior.

The library chooses configurations. It does not infer correct behavior, provide a test runner, or require users to describe their assertions in another language.

1. The small example should be the everyday interface

```
using UnitTestDesign
using Test
using Random

choices = (
    T = (Float32, Float64),
    storage = (:dense, :view),
    n = (0, 1, 17),
    pattern = (:zeros, :alternating, :random),
)

cases = all_pairs(choices)

@testset "my_sum $case" for case in cases
    rng = Xoshiro(1234)
    A = make_input(case; rng) # A fresh input for this configuration.
```

```
@test my_sum(A) == reference_sum(A)
end
```

Here `make_input`, `my_sum`, and `reference_sum` belong to the user's tests. The example assumes an appropriate reference and comparison for those inputs.

Each case is a `NamedTuple` with the same names and order as choices:

```
(T = Float32, storage = :view, n = 0, pattern = :zeros)
```

Names describe test dimensions, which need not correspond one-to-one with function arguments. A case can describe how to construct an array, an entire simulation, or a file. An author can pass `case...` as keywords when those names do match a function, but that is not required.

Keep the familiar generation functions:

```
all_values(choices)
all_pairs(choices)
all_triples(choices)
all_tuples(choices; n_way = 4)
full_factorial(choices)
```

`all_pairs` means that every feasible pair of factor values appears together in at least one returned case. It does not mean every pair of complete test cases, balanced frequencies, a minimum-size suite, or coverage of every program path.

2. Introduce a model object only when there is something to reuse

Simple calls accept a named tuple directly. `TestSpace` packages the same choices with constraints for repeated generation and inspection:

```
space = TestSpace(choices)
cases = all_pairs(space)
```

`all_pairs(choices)` is shorthand for `all_pairs(TestSpace(choices))`.

`TestSpace` contains choices and validity rules. Coverage strength, mandatory cases, and algorithm selection belong to the generation request. The same space can therefore support pairwise CI tests and a stronger release suite.

Domains are finite, nonempty, ordered collections. Singleton domains are allowed; users should not have to remove fixed settings from a model. `all_values` supports a one-factor model. A requested strength greater than the number of factors is a clear validation error. Empty models and empty domains are errors in this first interface.

Values can be Julia types, symbols, numbers, tuples, or other Julia objects. Selecting a value must preserve its type. Domain lookup and exact-pattern matching require both the same concrete type and `isequal` values. Thus `Int8(1)`, `Int64(1)`, and `1.0` can be distinct test choices. Duplicate choices under that relation are rejected with a useful explanation. Internally, generation works with choice indices rather than requiring values to be sortable.

The model copies the domain containers and does not mutate caller configuration. Selected mutable values are not automatically deep-copied. For tests that mutate inputs, choose descriptions

or factories and construct fresh fixtures per case. Mutating a domain value after constructing the model is outside the contract.

3. Constraints describe complete cases

The user-facing rule is:

Describe which complete configurations are allowed. You never need to handle a placeholder for an input that the generator has not chosen yet.

There are two primary forms: forbidden patterns and required relationships.

Forbidden patterns

```
space = TestSpace(  
    (  
        format = (:text, :binary),  
        compression = (:none, :gzip),  
        mode = (:buffered, :streaming),  
        seekable = (false, true),  
    );  
    rules = [  
        forbid(  
            (format = :text, compression = :gzip);  
            reason = "This text format does not support gzip",  
        ),  
        forbid(  
            (mode = :streaming, seekable = true);  
            reason = "Streaming mode does not support seeking",  
        ),  
    ],  
)
```

Within one pattern, all listed choices must match for it to forbid a case. Factors omitted from the pattern can take any value. Matching any forbidden pattern is enough to reject a case.

Format	Compression	First rule
:text	:none	Allows
:text	:gzip	Forbids
:binary	:none	Allows
:binary	:gzip	Allows

Patterns use exact values. A tuple-valued choice is still one value, not shorthand for alternatives. More complicated conditions use a predicate. The pattern is a positional named tuple so that meta-data such as reason cannot collide with a factor named reason.

Required relationships

```
square = require(:, rows, :columns); reason = "Matrix must be square") do rows, columns
    rows == columns
end

space = TestSpace(
    (rows = (0, 1, 8), columns = (0, 1, 8), T = (Float32, Float64));
    rules = [square],
)
```

The callback receives concrete values in the order of the named factors. All required relationships must return true for an allowed case. Julia’s do-block syntax puts the predicate first in the underlying function call:

```
require(predicate, (:rows, :columns); reason = "Matrix must be square")
```

Predicates must be deterministic and return `Bool`. They can be evaluated more than once, in an unspecified order. They should not mutate values or depend on changing external state. A thrown exception is reported as a rule-evaluation error, with its original cause and argument values; it does not silently mean “forbidden.”

An escape hatch accepts an ordinary complete-case predicate:

```
rule = require(; reason = "Configuration fits the memory budget") do case
    estimated_bytes(case) <= memory_budget
end
```

The estimate and budget are user code. This form receives a complete named tuple. Small scopes are preferable when natural because they permit earlier pruning and more useful explanations. They are not a prerequisite for correctness.

There is no public partial-value sentinel. `nothing` and `missing` may themselves be real domain values; their interpretation belongs to the user’s rule. A predicate returning `missing` instead of `Bool` is an error.

What constraints mean for coverage

A combination requires coverage if and only if it occurs in at least one complete case satisfying every rule.

For example, suppose *A*, *B*, and *C* each have choices (1, 2), with rules *A* == *B* and *B* == *C*. The only complete cases are (1, 1, 1) and (2, 2, 2). The pair *A* = 1, *C* = 2 is infeasible and does not need coverage. The user does not need to supply the implied relationship *A* == *C*.

The implementation must reason about valid completions. Merely finding no immediate violation in a partial case is not sufficient. Likewise, generating unconstrained cases and deleting forbidden rows cannot establish constrained coverage: removing a row can erase the only occurrence of another feasible pair.

This semantic promise is stronger than the current constraint implementation. It is an implementation requirement, not something a nicer callback fixes by itself. Backtracking over finite domains

is a possible starting point; a solver could support larger models or a restricted declarative rule language later. Arbitrary Julia predicates are not promised automatic translation into a solver.

If a search limit prevents establishing feasibility or completing a suite, the operation reports that limit. It must not label an unresolved combination infeasible or return a successful result claiming full coverage.

Invalid inputs can still be valuable tests

Rules define the population for this particular test. They do not describe every input the application might receive.

For a numerical-correctness test, exclude configurations the operation does not support. For a rejection test, deliberately generate those configurations and assert the documented exception. The documentation should show both uses so that “forbidden” does not accidentally become “never test this error path.”

4. Let users inspect rules without generating a suite

```
isallowed(space, complete_case)
explain(space, complete_case)
explain(space, (format = :text, compression = :gzip))
```

`isallowed` accepts only complete cases. `explain` also accepts partial assignments and checks whether a valid completion exists. It returns a structured explanation whose display is readable in the REPL.

Illustrative output:

No valid completion for:

```
format = :text
compression = :gzip
```

Conflicts with:

```
This text format does not support gzip
```

The result distinguishes `:allowed`, `:forbidden`, `:completable`, `:infeasible`, and `:unknown`. A completable partial assignment includes one witness completion. `:unknown` means a search limit prevented a conclusion.

A directly violated rule can be named immediately. An explanation involving several interacting rules may be less specific; the first implementation need not find a smallest conflicting subset. It must not invent a causal explanation. Unknown names and out-of-domain values are input errors, not constraint failures.

Model construction checks names, domains, and rule structure. It does not imply a potentially expensive proof that the whole model is satisfiable. Generating from an unsatisfiable model produces a model-level error, not an empty successful suite or an indexing error.

5. Make generated cases ordinary to consume and coverage explicit to inspect

Named generation returns a `TestCases` collection supporting `length`, iteration, and indexing. Its elements are named tuples. The collection retains its model and generation request, including algorithm information, so users can inspect what it was intended to cover. It does not track test execution.

```
cases = all_pairs(space)
cases[1]
collect(cases)           # Ordinary vector of named tuples.
report = coverage(cases)
```

`coverage(cases)` independently measures the returned rows against the stored request. For cases from other sources, the request is explicit:

```
report = coverage(space, existing_cases; n_way = 2)
```

A report includes:

- Requested interaction strength and any stronger groups.
- Number of rows examined and any invalid rows.
- Feasible interactions required, covered, and uncovered, when established.
- A bounded sample of uncovered interactions, expressed using factor names.
- Whether checking completed, and whether full coverage was established.

An invalid row contributes no coverage. A valid model with incomplete rows is not reported complete merely because those rows happened to cover some projections. If checking reaches a resource limit, the report exposes unresolved work and does not manufacture an exact percentage. Explicit coverage verification can itself be expensive; printing a collection must not silently perform it.

This operation measures the supplied configurations, not passed assertions, code coverage, or proven correctness. A user who skips cases during execution must pass the cases actually exercised to measure executed configuration coverage.

The small default display should show row count, factor names, and requested strength. It should not print a wall of internal matrices or unverified metrics.

6. Build on tests the user already values

```
regressions = [
  (T = Float32, storage = :view, n = 0, pattern = :zeros),
]
```

```
cases = all_pairs(choices; seeds = regressions)
```

Seeds are complete mandatory cases. They are checked against the domains and rules before generation and retained in supplied order, followed by additional cases. Invalid seeds produce an explanation; they are never silently discarded. Duplicate seed configurations are retained, preserving the user's requested list, but count only once toward distinct interaction coverage. Repetition for statistical testing is ordinarily better expressed in the execution loop.

This supports a useful workflow without a separate suite-management subsystem:

1. Record the configurations of existing tests.
2. Inspect their coverage.
3. Supply them as seeds to generate a completed suite.

Completion here means achieving the requested interaction coverage. It does not promise the mathematically smallest possible number of additions.

7. Expose stronger coverage without positional bookkeeping

```
cases = all_pairs(
    choices;
    extra = [strength(3, (:T, :storage, :n))],
)
```

This requests pairwise coverage globally and three-way coverage within the named group. `strength(3, (:a, :b, :c, :d))` would cover every feasible triple within that four-factor group. Group requirements are combined by union; overlapping requirements do not multiply obligations.

Names must exist, names within a group must be distinct, and a group's strength cannot exceed its size. The base and extra requirements must be represented in the coverage report. Generating cases must not modify the supplied groups.

Keep algorithm selection optional and retain the existing engine types:

```
cases = all_pairs(space; engine = IPOG())
cases = all_pairs(space; engine = GND(rng = Xoshiro(1234)))
```

The default should be deterministic for a fixed model and implementation version. Exact ordering across future versions is not a promise. Explicitly saved cases are stronger regression records than a generator seed alone.

An engine that cannot satisfy the requested semantics must report that limitation or use a documented correct fallback. Choosing a different engine must not weaken the meaning of “all feasible pairs.”

8. Keep excursions visibly distinct

```
cases = pairs_excursion(space)
```

This explores configurations differing from the baseline in at most two factors. The baseline is the first value in each domain, matching the existing convention. Filtering those excursions by the model's rules does not promise global pairwise coverage. An invalid baseline should be reported clearly.

For named models, use the explicit excursion functions. Do not encourage `all_pairs(space; engine = Excursion())`, because that spelling suggests the same coverage contract as the covering generators. Existing positional behavior can remain for compatibility while its different semantics are documented.

9. Fit randomized and property-based tests through ordinary composition

A case can choose a category such as `:near_zero` or `:ill_conditioned`. User code generates concrete values within that category and runs a property or reference comparison. More repetitions explore additional examples under the same configuration coverage.

This first interface needs no property-testing dependency or custom assertion macro. It also makes no promise to shrink concrete failures. If a property-testing integration proves useful, it should reuse the named configurations and coverage semantics rather than introduce a second model language.

Human users and AI assistants receive the same interface: normal Julia data, structured errors, named uncovered interactions, and ordinary test code. No AI-specific API is required. Factor selection and assertions remain explicit assumptions that can be reviewed.

10. Compatibility and a deliberately limited first implementation

Retain existing positional calls such as:

```
all_pairs([1, 2, 3], ["low", "high"], [false, true])
```

They retain their existing return convention. Dispatch on `NamedTuple` or `TestSpace` selects the new named interface. The old `disallow` callback is a legacy interface with partial-input semantics; it must not silently acquire new meaning. The proposed `rules` interface is separate and explicitly documented. Similarly, named groups replace positional wayness only in the new interface.

The initial surface is:

Operation	Purpose
<code>TestSpace</code>	Reusable finite choices and validity rules
<code>forbid, require</code>	Exact forbidden patterns and ordinary Julia relationships
Existing generation names	Select cases at a requested strength
<code>strength</code>	Describe an additional coverage group by name
<code>isallowed, explain</code>	Inspect validity and possible completions
<code>coverage</code>	Measure coverage of generated or existing configurations

The first implementation should omit a custom runner, macro language, automatic model inference, CLI, general serialization framework, distributed scheduling, automatic shrinking, and a large catalog of domain-specific fixtures. These may be useful later, but none is required to test the central interface hypothesis.

Questions for competing proposals

These are the decisions on which an alternative could reasonably do better:

1. Is a named overload of `all_pairs` easier to discover than a new entry point such as `covering_cases`? The proposal favors familiarity over a fresh naming scheme.

2. Are `forbid` patterns and `scoped require` predicates worth two concepts, or would one complete-case predicate be enough? The proposal favors readable common exclusions while keeping arbitrary Julia available.
3. Is `TestCases` with retained metadata worth more complexity than returning a plain vector? The proposal favors inspectability, with `collect` as the exit.
4. Can correct constrained generation be acceptably fast with ordinary Julia predicates? If not, should the package limit supported constraints, introduce a declarative subset, or make the cost of opaque predicates more visible?
5. Is coverage inspection and completion useful enough to justify building a model? It still requires the user to identify factors; the proposal does not remove that cost.
6. Does this interface improve a real test suite enough that its author would keep it after comparing exhaustive loops and random selection under the same budget?

The strongest first demonstration would use an existing expensive Julia test: express its choices and rules, measure existing coverage, complete the missing interactions, and run the existing assertions. The evaluation should include authoring effort, execution cost, and fault detection. A smaller row count alone would not establish that the interface is worth adopting.